# MA22037. Felix Galka (fg551)

# Gen-AI Statement

I used Generative AI for the following things within this notebook:

- Finding previously unused LaTeX symbols, such as $[\dots]_0^1$ and piecewise function definition in LaTeX (A2)
- Explaining unseen concepts such as analytic functions and the tail sum formula $\sum_{k>J}^{\infty} f(k) \leq \int_J^{\infty} f(x)dx$, when online resources weren't explaining well enough.
- Ideas on how to present matplotlib plots more visually appealing (legends, font customisation)
- General code debugging

# Section A

## A1

**a)**

By using these facts,

$$|f(x)|^2 = f(x) \cdot \overline{f(x)}, \quad f(x) = \sum_{k=\infty}^{\infty} u_k e^{2k\pi ix}, \quad \overline{f(x)} = \sum_{j=\infty}^{\infty} \overline{u_j e^{2j\pi ix}}$$

We can prove the following about $|f(x)|^2$

$$\begin{aligned} |f(x)|^2 &= \left(\sum_{k=\infty}^{\infty} u_k e^{2k\pi ix}\right)\left(\sum_{j=\infty}^{\infty} \overline{u_j e^{2j\pi ix}}\right) \\ &= \left(\sum_{k=\infty}^{\infty} u_k e^{2k\pi ix}\right)\left(\sum_{j=\infty}^{\infty} \overline{u_j} e^{-2j\pi ix}\right) \end{aligned}$$

Since both sums converge finitely according to the Section A preamble, the sums can be merged

$$\begin{aligned} |f(x)|^2 &= \sum_{k=\infty}^{\infty}\sum_{j=\infty}^{\infty} u_k e^{2k\pi ix}\overline{u_j} e^{-2j\pi ix} \\ &= \sum_{k=\infty}^{\infty}\sum_{j=\infty}^{\infty} u_k \overline{u_j} e^{2(k-j)\pi ix} \end{aligned}$$

Which can then be plugged in as follows

$$\begin{aligned}\int_0^1 |u(x)|^2\,dx &= \int_0^1 \sum_{k=\infty}^{\infty}\sum_{j=\infty}^{\infty} u_k\overline{u_j}e^{2(k-j)\pi ix}\,dx \\ &= \sum_{k,j=-\infty}^{\infty}\int_0^1 u_k\bar{u}_j e^{2\pi ix(k-j)}\,dx \\ &= \sum_{k,j=-\infty}^{\infty} u_k\bar{u}_j\int_0^1 e^{2\pi ix(k-j)}\,dx \qquad \mathbf{(1)}\end{aligned}$$

Now, we prove that the integral in $\mathbf{(1)}$ equals the Kronecker delta $\delta_{j,k}$

$$\begin{aligned}\int_0^1 e^{2\pi ix(k-j)}\,dx &= \left[\frac{e^{2\pi i(k-j)x}}{2\pi i(k-j)}\right]_0^1 \\ &= \frac{1}{2\pi i(k-j)}\left[e^{2\pi i(k-j)}-1\right]\end{aligned}$$

Since $(k-j)\in\mathbb{Z},\ \ e^{2(k-j)\pi i}=1$, and the integral in $\mathbf{(1)}$ = 0 for $k\neq j$.
For $k=j,\ \mathbf{(1)}$ becomes $\int_0^1 e^0\,dx=\int_0^1 1\,dx=1\ \ \square$

Back to the original proof now, the double sum simplifies as follows:

$$\begin{aligned}\int_0^1 |u(x)|^2\,dx &= \sum_{k,j=-\infty}^{\infty} u_k\bar{u}_j\delta_{k,j} \\ &= \sum_{k=-\infty}^{\infty} u_k\bar{u}_k \\ &= \sum_{k=-\infty}^{\infty} |u_k|^2 \ \ \square\end{aligned}$$

Since $v\bar{v}=|v|^2$

**b)**

Using Hint 2,

$$\begin{aligned}u(x)\times e^{-2k\pi ix} &= \sum_{j=-\infty}^{\infty} u_j e^{2j\pi ix}\times e^{-2k\pi ix} \\ &= \sum_{j=-\infty}^{\infty} u_j e^{2(j-k)\pi ix}\end{aligned}$$

Now, integrating over [0,1],

$$\begin{aligned}\int_0^1 \sum_{j=-\infty}^{\infty} u_j e^{2(j-k)\pi ix}\,dx &= \sum_{j=-\infty}^{\infty} u_j\int_0^1 e^{2(j-k)\pi ix}\,dx \\ &= \sum_{j=-\infty}^{\infty} u_j\delta_{j,k} \qquad from\ \mathbf{(1)} \\ &= u_k\ \ \square\end{aligned}$$

# A2

We can use the result from A1 part b) to directly compute the Fourier coefficients for $u(x)$

$$\begin{aligned} u_k &= \int_0^1 u(x) \times e^{-2\pi kix}\, dx \\ &= \int_0^{1/2} 0 \times e^{-2\pi kix}\, dx + \int_{1/2}^1 1 \times e^{-2\pi kix}\, dx \\ &= \int_{1/2}^1 e^{-2\pi kix}\, dx \\ &= \frac{\left[e^{-2\pi ikx}\right]_{1/2}^1}{-2\pi ik} \\ &= \frac{\left[e^{-2\pi ik} - e^{-\pi ik}\right]}{-2\pi ik} \end{aligned}$$

However, $e^{-2\pi ki} = 1$ and $e^{-\pi ki} = (-1)^k$
$u_k$ thus simplifies to:

$$u_k = \frac{1 - (-1)^k}{-2\pi ik} = \frac{(-1)^k - 1}{2\pi ik}$$

Considering the case k = 0, we observe

$$u_0 = \int_0^1 u(x) e^0\, dx = \frac{1}{2}$$

Finally,

$$u_k = \begin{cases} \frac{1}{2}, & \text{k=0} \\ 0, & k\ even \\ \frac{i}{\pi k}, & k\ odd \end{cases}$$

# A3

In [2]:
```
import numpy as np

# Note -> I've had to truncate a lot of lines using \
# as the PDF doesn't wrap text within code blocks

def eval_uj(J, x):

        # Calculate the u_k coefficients from k = -J to k = J
        # from the above formula

        uJ_x = 0
        for k in range(-J, J + 1):
                u_k = 0
                if k == 0:
                        u_k = 1/2
                elif k % 2 == 0:
                        u_k = 0
                else:
                        u_k = 1j / (np.pi * k)
                uJ_x += (u_k * np.exp(2 * np.pi * 1j * k * x))
        return uJ_x
```

```python
def step_fn(J):

        # Generate the x and y axis

        N=1000
        grid=np.linspace(0,1,N)
        uJ = []
        for point in grid:
                uJ.append(eval_uj(J, point).real)
        return grid, uJ

import matplotlib.pyplot as plt

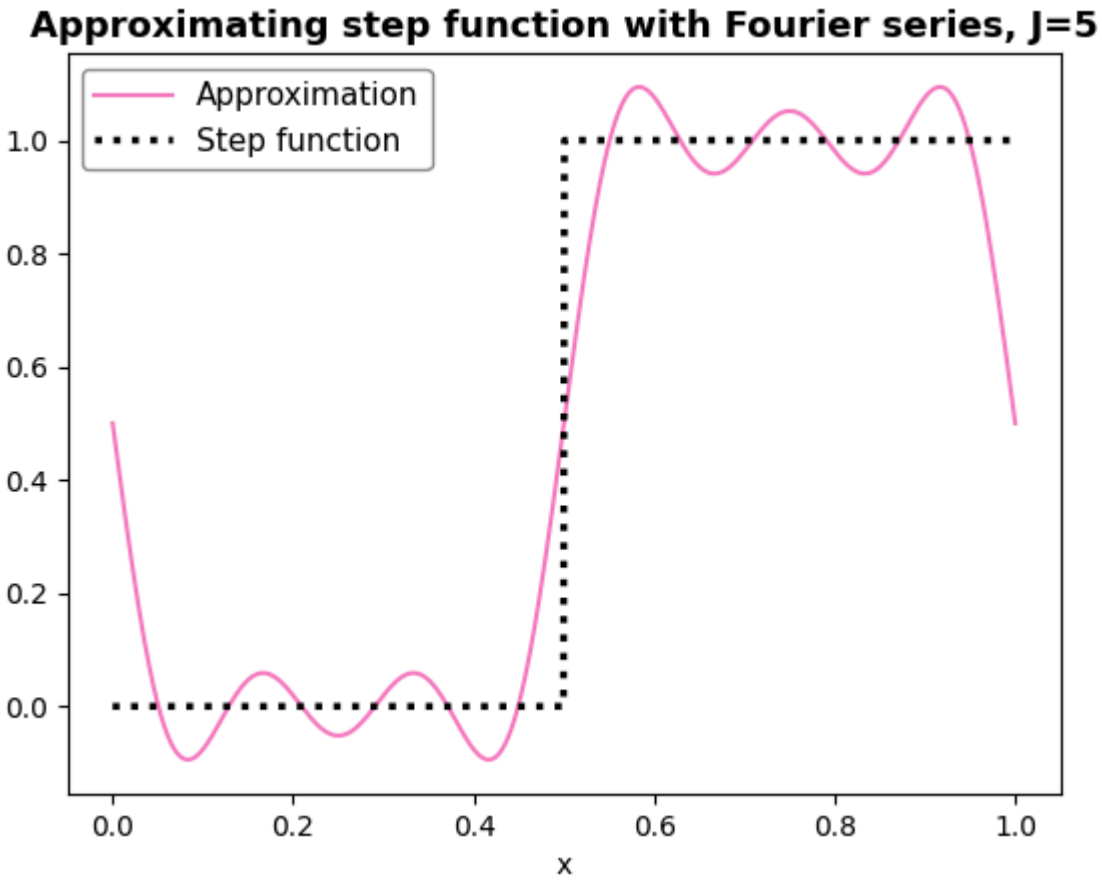
step = [0 if x < 1/2 else 1 for x in np.linspace(0,1,1000)]

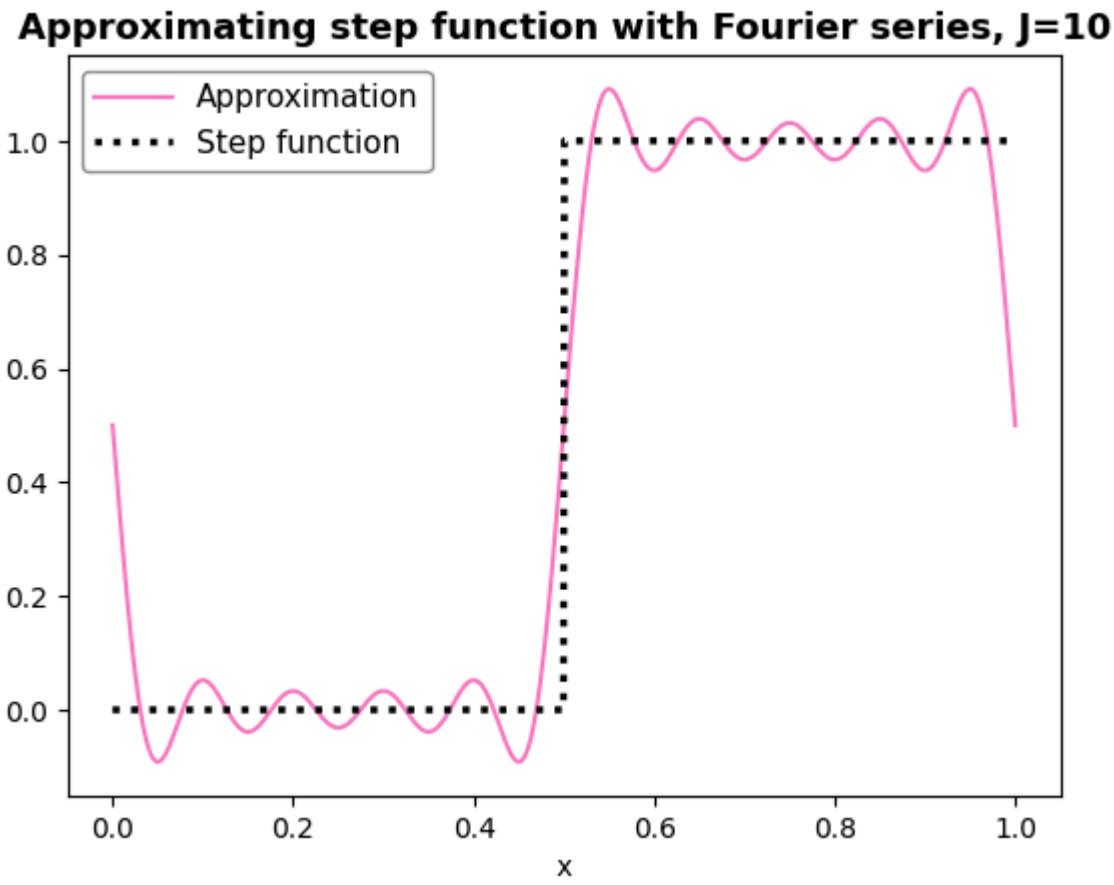
J = [5,10,25,50]

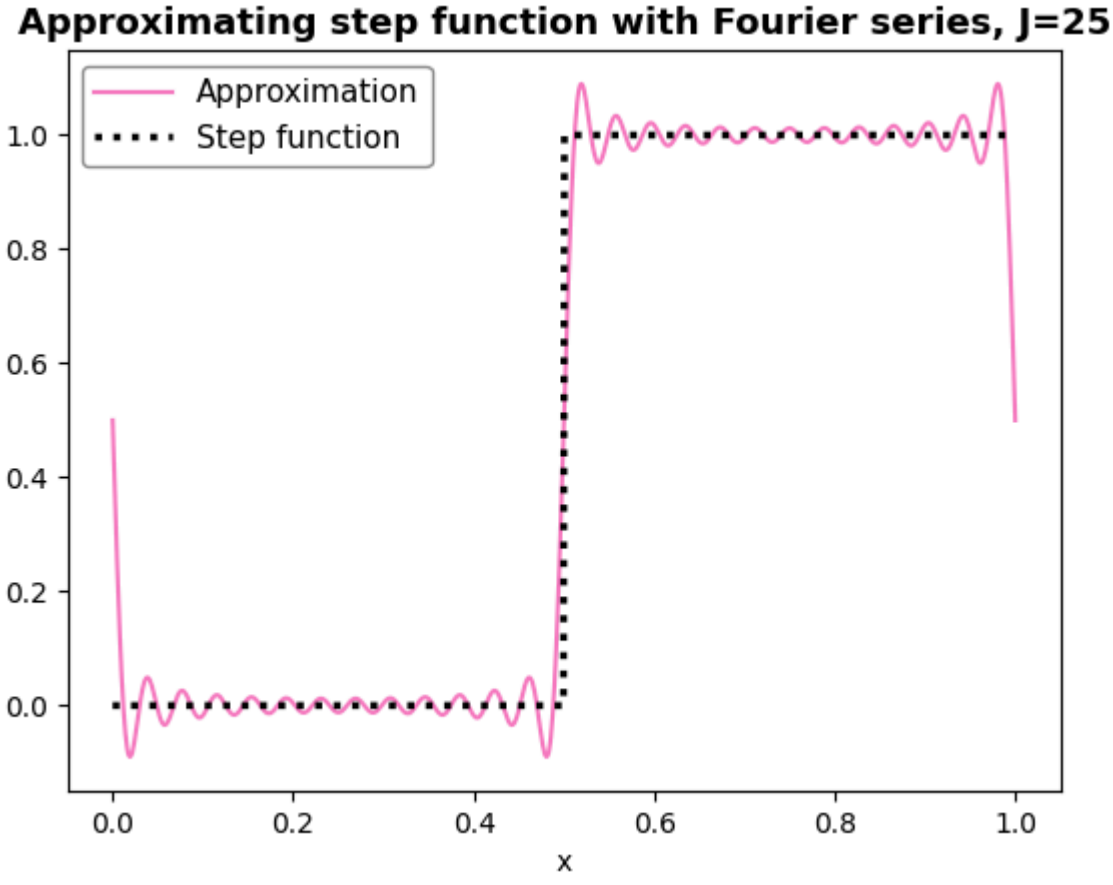
for j in J:
        x, y = step_fn(j)
        plt.plot(x, y, label=f"Approximation", color= "#f542a7", \
                alpha=0.7,linewidth=1.5)
        plt.plot(x, step, linestyle="dotted", label="Step function",\
                color="black", linewidth=2.5)

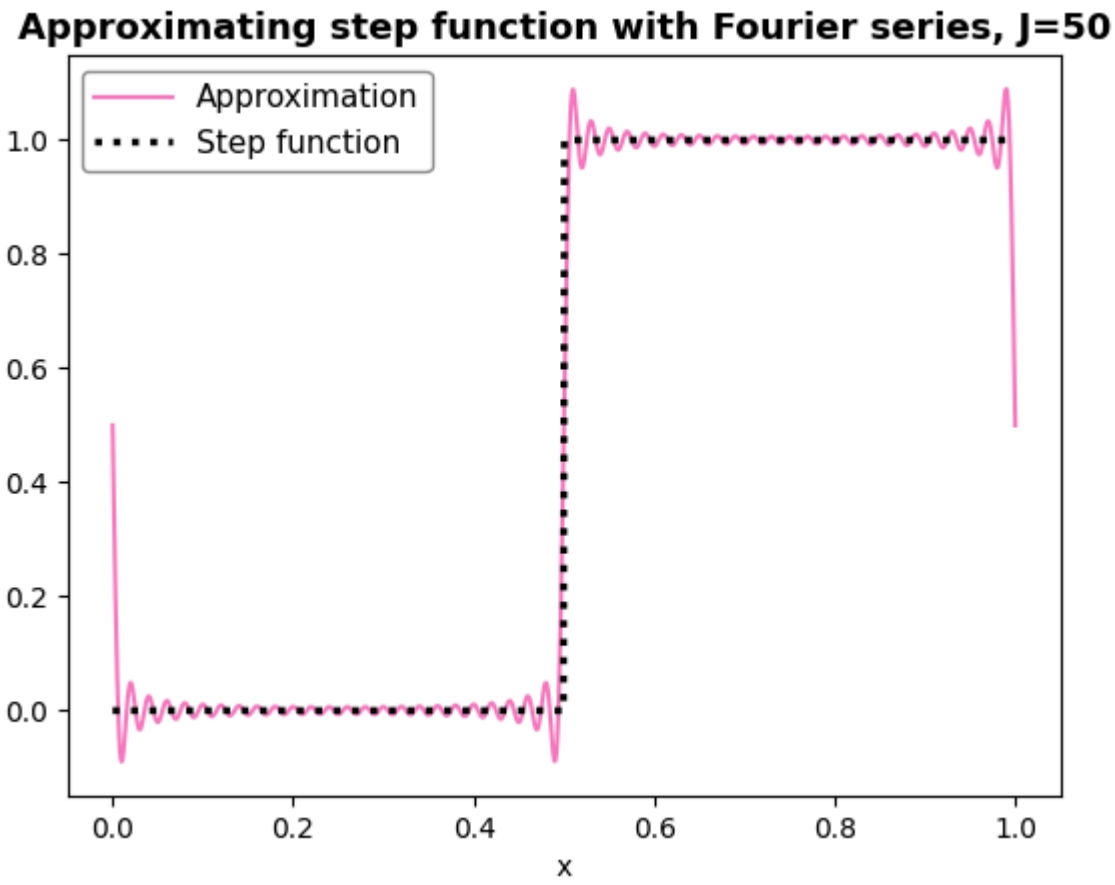
        plt.xlabel("x")
        plt.title(f"Approximating step function with Fourier series, J={j}"
                , fontweight="bold", fontsize=13)
        plt.legend(framealpha=0.9, edgecolor='grey', fontsize=11,handlelength=2.5); plt.show(

print()
```




Approximating step function with Fourier series, J=10
Approximation
Step function
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
x


Approximating step function with Fourier series, J=25
Approximation
Step function
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
x




## Main features of approximation

- The approximation gets more accurate the further away it is from discontinuous points / end points.
- The approximation is worst at the end points, with massive errors occuring as $x \to 1$ and $x \leftarrow 0$.
- Even comparing $J = 5$ and $J = 50$, the scale of the errors at $x = 0$ and $x = 1$ stay the same, they just happen closer to the endpoint as J increases.
- As J increases, the peaks and trophs of the sin/cos oscillations get smaller in magnitude, thus approximating the line better.

## Gibbs' Phenomenon

Gibbs' Phenomenon occurs when approximating discontinuous functions with oscillating functions, not unique to Fourier series but commonly found in such approximations.
At discontinuous points ($x = 1/2$ for example), Gibbs' Phenomenon says that the approximation will overshoot by roughly 9%. At the endpoints, the Fourier approximation expects something similar to the jump at $x = 1/2$, so it overshoots massively. This is due to the periodicity of the approximation, and explains the symmetry of the pink curve at $x = 1/4, 3/4$. The jump at $x = 1/2$ has been essentially encoded into the sin/cos oscillations and cannot be forgotten, thus the repetition at the endpoints, which are the same distance away from the jump.

References


- *https://ocw.mit.edu/courses/18-03sc-differential-equations-fall-2011/05cce833730ffd3c39f420a41ad82fd6_MIT18_03SCF11_s22_7text.pdf*
- *https://math.libretexts.org/Bookshelves/Differential_Equations/Introduction_to_Partial_Differential_Equatio*
- *https://en.wikipedia.org/wiki/Gibbs_phenomenon*

# A4

Repeating the above process for $u(x) = x^2 - x$,

$$\begin{aligned} u_k &= \int_0^1 u(x) \times e^{-2\pi kix}\, dx \\ &= \int_0^1 (x^2 - x) e^{-2\pi kix}\, dx \\ &= \int_0^1 x^2 e^{-2\pi kix}\, dx \; \mathbf{(2)} - \int_0^1 x e^{-2\pi kix}\, dx \; \mathbf{(3)} \end{aligned}$$

Using integration by parts with $f(x) = x^2$ and $g(x) = e^{-2\pi kix}$, and assuming $k \neq 0$,

$$\begin{aligned} \mathbf{(2)} &= \left[\frac{x^2 e^{-2k\pi ix}}{-2k\pi i}\right]_0^1 + \frac{1}{\pi ki}\int_0^1 x e^{-2k\pi ix}\, dx \\ &= \frac{-1}{2k\pi i} + \frac{1}{\pi ki}\int_0^1 x e^{-2k\pi ix}\, dx \\ &= \frac{-1}{2k\pi i} + (\frac{1}{\pi ki} \times \mathbf{(3)}) \end{aligned}$$

Again, with integration by parts with $f(x) = x$ and $g(x) = e^{-2\pi kix}$

$$\begin{aligned} \mathbf{(3)} &= \left[\frac{x e^{-2\pi ikx}}{-2\pi ki}\right]_0^1 + \frac{1}{2\pi ki}\underbrace{\int_0^1 e^{-2k\pi ix}\, dx}_{= 0 \; as \; proven \; in \; A1} \\ &= \frac{1}{-2k\pi i} \end{aligned}$$

Substituting $\mathbf{(3)}$ into $\mathbf{(2)}$:

$$\begin{aligned} \mathbf{(2)} &= -\frac{1}{2k\pi i} - \frac{1}{2\pi^2 k^2 i^2} \\ &= \frac{1}{2\pi^2 k^2} - \frac{1}{2k\pi i} \end{aligned}$$

Finally, substituting these into the expression for $u_k$:

$$\begin{aligned} u_k &= \mathbf{(2)} - \mathbf{(3)} \\ &= \frac{1}{2\pi^2 k^2} \end{aligned}$$

Looking at the k=0 case carefully:

$$\begin{aligned} u_0 &= \int_0^1 u(x) e^0\, dx \\ &= \int_0^1 x^2 - x\, dx \qquad = \left[\frac{x^3}{3} - \frac{x^2}{2}\right]_0^1 \\ &= -\frac{1}{6} \end{aligned}$$

Now that we have all cases covered, the final definiton for $u_k$ is as follows:

$$u_k = \begin{cases} -\frac{1}{6}, & k = 0 \\ \frac{1}{2\pi^2 k^2}, & k \neq 0 \end{cases}$$

In [3]:

```
import numpy as np
import matplotlib.pyplot as plt

# Note -> I've had to truncate a lot of lines using \
# as the PDF doesn't wrap text within code blocks

def eval_uj(J, x):

        uJ_x = 0
        for k in range(-J, J + 1):
                u_k = 0
                if k == 0:
                        u_k = - 1/6
                else:
                        u_k = 1 / ( (np.pi **2) * (k**2) * 2)
                uJ_x += (u_k * np.exp(2 * np.pi * 1j * k * x))
        return uJ_x

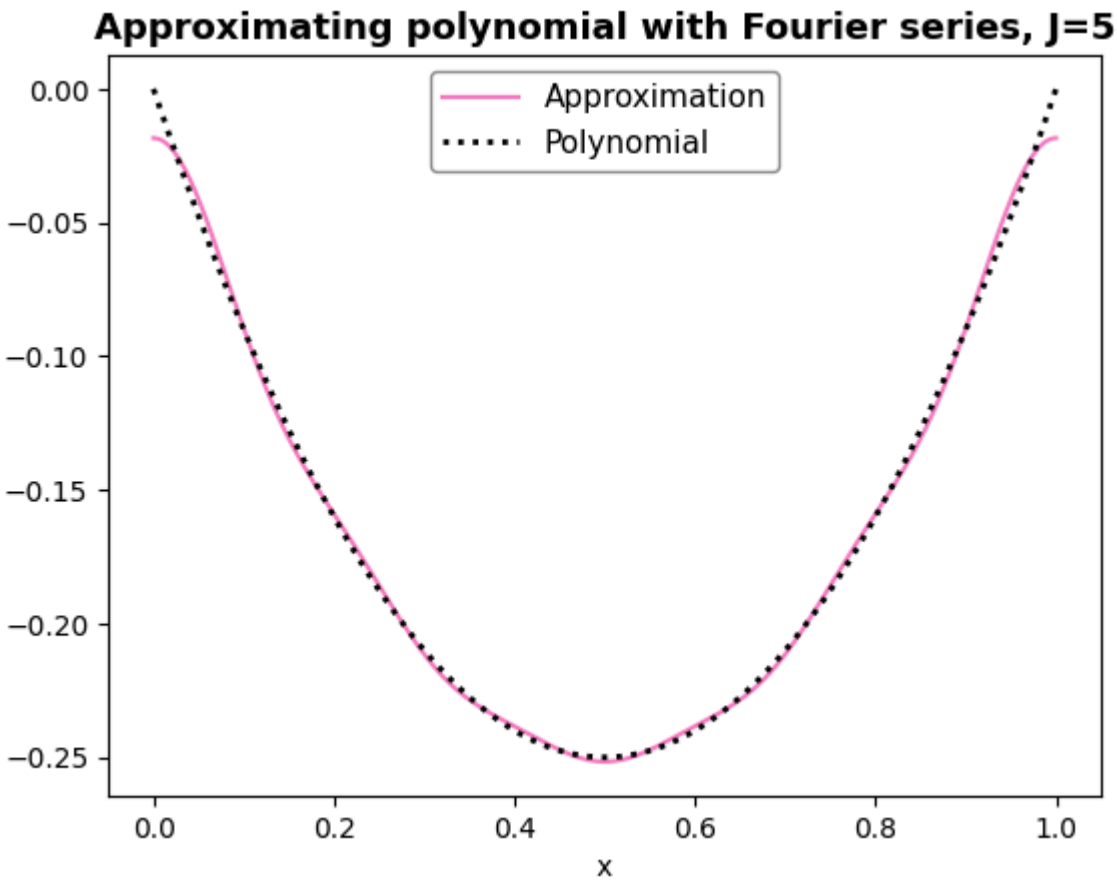
def poly_fn(J):
        N=1000
        grid=np.linspace(0,1,N)
        uJ = []
        for point in grid:
                uJ.append(eval_uj(J, point).real)
        return grid, uJ

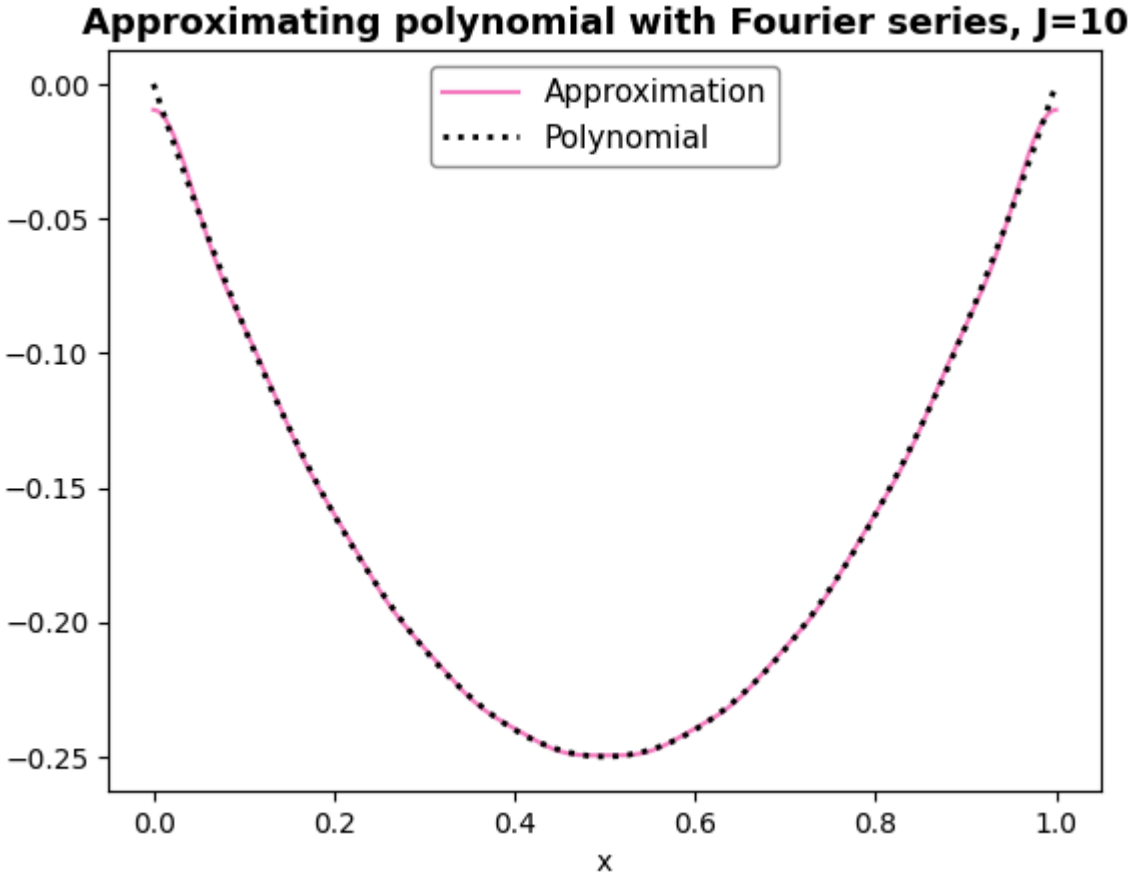
u = [x**2 - x for x in np.linspace(0,1,1000)]

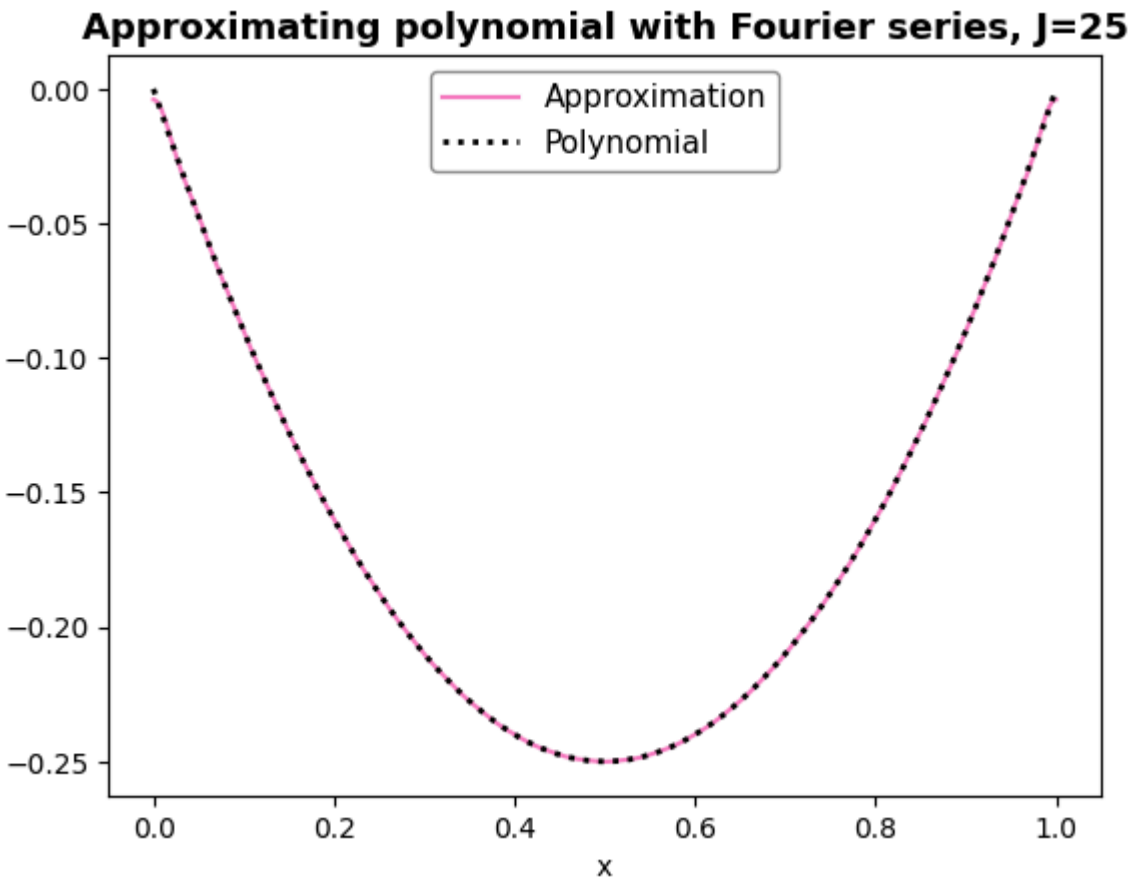
J = [5,10,25,50]

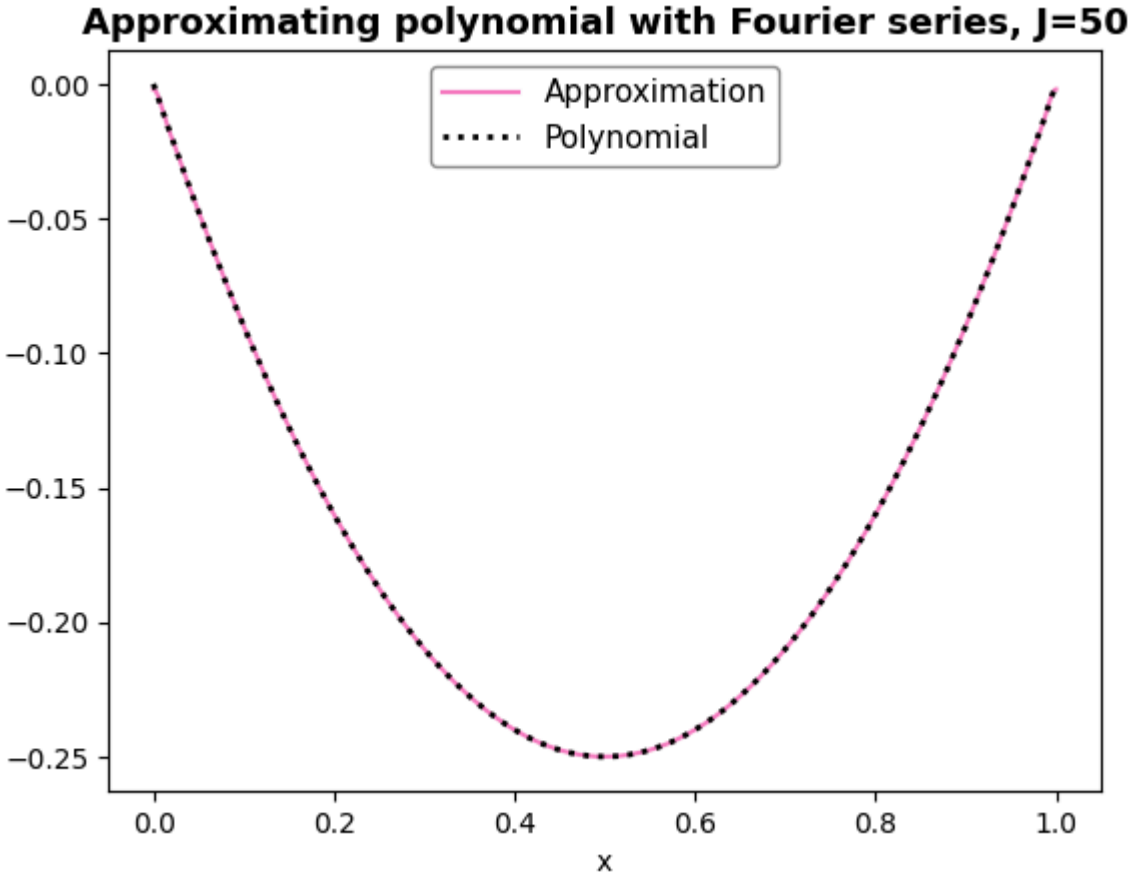
for j in J:
        x, y = poly_fn(j)
        plt.plot(x, y, label=f"Approximation", color= "#f542a7", \
                alpha=0.7, linewidth=1.5)
        plt.plot(x, u, linestyle="dotted", label="Polynomial",color="black", linewidth=2)
        plt.xlabel("x"); plt.title(f"Approximating polynomial with Fourier series, J={j}",\
                fontweight="bold", fontsize=13)
        plt.legend(framealpha=0.9, edgecolor='grey', fontsize=11,handlelength=2.5);
        plt.show()

print()
```


Approximating polynomial with Fourier series, J=5
Approximation
Polynomial
0.00
−0.05
−0.10
−0.15
−0.20
−0.25
0.0
0.2
0.4
0.6
0.8
1.0
x


Approximating polynomial with Fourier series, J=10
Approximation
Polynomial
0.00
−0.05
−0.10
−0.15
−0.20
−0.25
0.0
0.2
0.4
0.6
0.8
1.0
x




## A4 Commentary

- Comparing to the step function, the Fourier approximation almost perfectly approximated $u(x)$ by even $J = 25$, with already substantial progress made by $J = 5$.
- As J increases, we can see a dampening in the oscillations again, with the tails almost perfectly matching at the endpoints with $J = 50$.

- It is clear that this function doesn't cause Gibbs' Phenomenom to occur, as it is a continuous, smooth polynomial
- The smoothness of this function on [0,1] compared to the previous step function means no large oscillations need to be encoded at any point on the interval, explaining why the approximation is so great even for low J, as there isn't any jumpy behavior for the encoded oscillations to account for.

# A5

**a)**

Proceeding with integration by parts of $u_j = \int_0^1 u(x)e^{-2\pi jix}\,dx$ with $f(x) = u(x),\ g(x) = e^{-2j\pi ix}$:

$$\begin{aligned} u_j &= \left[\frac{u(x)e^{-2j\pi ix}}{-2j\pi i}\right]_0^1 + \frac{1}{2j\pi i}\int_0^1 u'(x)e^{-2j\pi ix}\,dx \\ &= \frac{1}{-2j\pi i}[u(1)-u(0)] + \frac{1}{2j\pi i}\int_0^1 u'(x)e^{-2j\pi ix}\,dx \end{aligned}$$

Now, taking the absolute value of both sides

$$\begin{aligned} |u_j| &= \left|\frac{u(1)-u(0)}{2\pi j}\right| + \left|\frac{1}{2j\pi}\int_0^1 u'(x)e^{-2j\pi ix}\,dx\right| \\ &\le \frac{1}{2\pi|j|}\left(|u(0)-u(1)| + \int_0^1 \left|u'(x)e^{-2j\pi ix}\right|\,dx\right) \quad \textbf{(4)} \\ &\le \frac{1}{2\pi|j|}\left(|u(0)-u(1)| + \int_0^1 |u'(x)|\,dx\right) \quad \textbf{(5)} \\ &\le \frac{C_1}{|j|}, \quad C_1 = \frac{1}{2\pi}\left(|u(0)-u(1)| + \int_0^1 |u'(x)|\,dx\right) \quad \square \end{aligned}$$

, with $\le$ introduced in $\textbf{(4)}$ due to this inequality: $\left|\int_a^b f(x)\,dx\right| \le \int_a^b |f(x)|\,dx$
and with $\textbf{(5)}$ using the fact that $e^{-2j\pi ix} \le 1 \ \therefore\ u'(x)e^{-2j\pi ix} \le u'(x)$

**b)**

To prove $\int_0^1 (u(x)-\tilde{u}_J(x))^2\,dx \le \frac{2C_1^2}{J-1}$, start by defining $e_J(x)$ as the error:

$$\begin{aligned} e_J(x) &= u(x) - \tilde{u}_J(x) \\ &= \sum_{|k|>J} u_k e^{2k\pi ix} \end{aligned}$$

Now, we plug this into the above and use a very similar argument to A1 with different bounds on the summations, hence I won't make this proof as verbose.

$$
\begin{aligned}
\int_0^1 |e_J(x)|^2 \, dx &= \int_0^1 \left| \sum_{|k|>J} u_k e^{2k\pi ix} \right|^2 \, dx \\
&= \int_0^1 \left( \sum_{|k|>J} u_k e^{2k\pi ix} \right) \left( \sum_{|j|>J} \overline{u_j} e^{-2j\pi ix} \right) \, dx \\
&= \int_0^1 \sum_{|k|,|j|>J} u_k \overline{u_j} e^{2\pi ix(k-j)} \, dx \\
&= \sum_{|k|,|j|>J} u_k \overline{u_j} \int_0^1 e^{2\pi ix(k-j)} \, dx \\
&= \sum_{|k|,|j|>J} u_k \overline{u_j} \delta_{k,j} \\
&= \sum_{|k|>J} u_k \overline{u_k} \\
&= \sum_{|k|>J} |u_k|^2 \\
&= 2 \sum_{k>J} |u_k|^2
\end{aligned}
$$

The introduction of the *2* factor on the last line is with the following reasoning. We are summing over $(J, \infty)$ & $(-\infty, -J)$. For negative $k$, we sum over $|u_{-k}|^2$. But for real-valued functions $u(x)$,

$$
\begin{aligned}
u_{-k} &= \int_0^1 u(x) e^{2k\pi ix} \, dx \\
\overline{u}_k &= \int_0^1 \overline{u(x) e^{-2k\pi ix}} \, dx = \int_0^1 \overline{u(x)} e^{2k\pi ix} \, dx \\
&= \int_0^1 u(x) e^{2k\pi ix} \, dx \\
\overline{u}_k &= u_{-k}
\end{aligned}
$$

$$
\therefore |u_{-k}|^2 = |\overline{u}_k|^2 = |u_k|^2
$$

$$
\therefore \sum_{|k|>J} |u_k|^2 = 2 \sum_{k>J} |u_k|^2 \quad \square
$$

To complete the proof, substitute $u_k$ for the bound $C_1$ described above and use the fact that $\sum_{k>J} u(k) \leq \int_J^\infty u(x) \, dx$, for monotonically decreasing $u(x)$. This can be explained geometrically if you imagine the sum of rectangles formed by $u(k)$ points under the curve. These rectangles will always lie beneath the curve, thus the integral will be greater than the sum. Here $\frac{1}{x^2}$ is decreasing for positive $x > J$.

$$\int_0^1 |e_J(x)|^2 \, dx = 2\sum_{k>J} |u_k|^2 \quad \textbf{(6)}$$

$$\leq 2C_1^2 \sum_{k>J} \frac{1}{|k|^2}$$

$$\leq 2C_1^2 \int_J^\infty \frac{1}{x^2} \, dx$$

$$\leq 2C_1^2 \left| \frac{-1}{x} \right|_J^\infty$$

$$\leq 2C_1^2 \left( \frac{1}{J} \right)$$

and finally since $\frac{1}{J} < \frac{1}{J-1}$, we get:

$$\int_0^1 |e_J(x)|^2 \, dx \leq 2C_1^2 \frac{1}{J-1} \quad \square$$

**c)**

Taking a similar approach to **A5 a)**, we integrate $u_k$ by parts to find our $C_2$. From **a)** and the fact that $u(0) = u(1)$, the first integral becomes

$$u_k = \frac{1}{2k\pi i} \int_0^1 u'(x) e^{-2k\pi i x} \, dx$$

We then integrate by parts again with $f(x) = u'(x), g(x) = e^{-2k\pi i x}$ to get:

$$\begin{aligned} u_k &= \frac{1}{2k\pi i} \left| \frac{u'(x) e^{-2k\pi i x}}{-2k\pi i} \right|_0^1 + \frac{1}{2k\pi i} \left( \frac{1}{2k\pi i} \int_0^1 u''(x) e^{-2k\pi i x} \, dx \right) \\ &= \frac{1}{-(2k\pi i)^2} \big( u'(1) - u'(0) \big) + \frac{1}{(2k\pi i)^2} \int_0^1 u''(x) e^{-2k\pi i x} \, dx \\ &= \frac{1}{4k^2\pi^2} \big( u'(1) - u'(0) \big) + \frac{-1}{4k^2\pi^2} \int_0^1 u''(x) e^{-2k\pi i x} \, dx \end{aligned}$$

Taking absolute values of each side and using the exact same arguments as earlier to introduce the $\leq$ sign,

$$
\begin{aligned}
|u_k| &= \frac{1}{4k^2\pi^2}|u'(1) - u'(0)| + \left|\frac{-1}{4k^2\pi^2}\int_0^1 u''(x)e^{-2k\pi ix}\,dx\right| \\
&\leq \frac{1}{4k^2\pi^2}|u'(1) - u'(0)| + \frac{-1}{4k^2\pi^2}\int_0^1 \left|u''(x)e^{-2k\pi ix}\right|\,dx \\
&\leq \frac{1}{4k^2\pi^2}|u'(1) - u'(0)| + \frac{-1}{4k^2\pi^2}\int_0^1 |u''(x)|\,dx \\
&\leq \frac{1}{4k^2\pi^2}\left(|u'(1) - u'(0)| + \int_0^1 |u''(x)|\,dx\right)
\end{aligned}
$$

$$
|u_k| \leq \frac{C_2}{k^2}\ with\ C_2 = \frac{|u'(1) - u'(0)| + \int_0^1 |u''(x)|\,dx}{4\pi^2} \quad \square
$$

Now, we can repeat the exact same argument as before, starting from $(6)$ in **A5 b)**

$$
\begin{aligned}
\int_0^1 |e_J(x)|^2\,dx &= 2\sum_{k>J}|u_k|^2 \\
&\leq 2C_2^2\sum_{k>J}\frac{1}{|k^4|} \\
&\leq 2C_2^2\int_J^\infty \frac{1}{x^4}\,dx \\
&\leq 2C_2^2\left|\frac{1}{-3x^3}\right|_J^\infty
\end{aligned}
$$

$$
\int_0^1 |e_J(x)|^2\,dx \leq \frac{2C_2^2}{3J^3}
$$

Finally, letting $C_3 = \frac{2}{3}C_2^2$ and seeing again that $\frac{1}{(J-1)^3} > \frac{1}{J^3}$, we arrive to

$$
\int_0^1 |u(x) - \tilde{u}_J(x)|^2\,dx \leq \frac{C_3}{(J-1)^3} \quad \square
$$

Where crucially, $C_3$ depends on $u''(x)$.

**d)**

In A2, we use the step function which is non-periodic and not continuously differentiable on [0,1] due to the lack of continuity at $x = \frac{1}{2}$. Thus none of the proven error upper bounds apply for this choice of $u(x)$ and with the combination of Gibbs' phenomenon, the quality of approximation is extremely low.

However in A4, we have $u(x) = x^2 - x$ which is both periodic on [0,1] and twice continuously differentiable on [0,1], since it is a polynomial of degree 2.

Using the inequality

$$
\int_0^1 |u(x) - \tilde{u}_J(x)|^2\,dx \leq \frac{C_3}{(J-1)^3}
$$

We can see that the error's rate of convergence is $O(\frac{1}{J^3})$, which with the graphs shown above, approximates extremely well even for smaller J. This, combined with the relative smoothness of the choice of $u(x)$ (first & second derivative are relatively small), the approximation is extremely high quality.

# Section B

## B1

Let each point on the mesh [0,1] be considered [$\frac{j}{J},\ j = 0...J$ ]. Consider the first and last few trapeziums made by the rule and their areas $A_k,\ k = 1 \dots\ J$

$$
\begin{aligned}
A_1 &= h\frac{u(0) + u(1/J)}{2} \\
A_2 &= h\frac{u(1/J) + u(2/J)}{2} \\
&\dots \\
A_{J-1} &= h\frac{u(J-2/J) + u(\frac{J-1}{J})}{2} \\
A_J &= h\frac{u(\frac{J-1}{J}) + u(1)}{2}
\end{aligned}
$$

Remembering that $u(1) = u(0), h = 1/J$ and noticing that each $u(j/J)$ term is repeated twice, we get:

$$
\begin{aligned}
T(u) &= A_1 + A_2 + \ldots + A_{J-1} + A_J \\
&= \frac{h}{2}\left(u(0) + 2u(1/J) + \ldots + 2u\left(\frac{J-2}{J}\right) + 2u\left(\frac{J-1}{J}\right) + u(1)\right) \\
&= \frac{h}{2}\left(2u(0) + 2u(1/J) + \ldots + 2u\left(\frac{J-2}{J}\right) + 2u\left(\frac{J-1}{J}\right)\right) \\
&= \frac{1}{J}\left(u(0) + u(1/J) + \ldots + u\left(\frac{J-2}{J}\right) + u\left(\frac{J-1}{J}\right)\right) \\
&= \frac{1}{J}\sum_{j=0}^{J-1} u\left(\frac{j}{J}\right) \quad \square
\end{aligned}
$$

In [ ]:
```
from scipy.integrate import quad # Use quad to find real integrals to a good degree of accura

# Note -> I've had to truncate a lot of lines using \
# as the PDF doesn't wrap text within code blocks

def per_trap(u,J):
        # For periodic functions
        I = 0
        for j in range(0, J):
                I += u(j/J)
        return I/J

def non_per_trap(u,J):
        # Add the tails for non-periodic functions
        I = 0.5 * (u(0) + u(1))
        for j in range(1,J):
                I += u(j/J)
        return I/J
```

```
J = [i for i in range(1,50)]

errs_exp = []
errs_cos = []

for j in J:

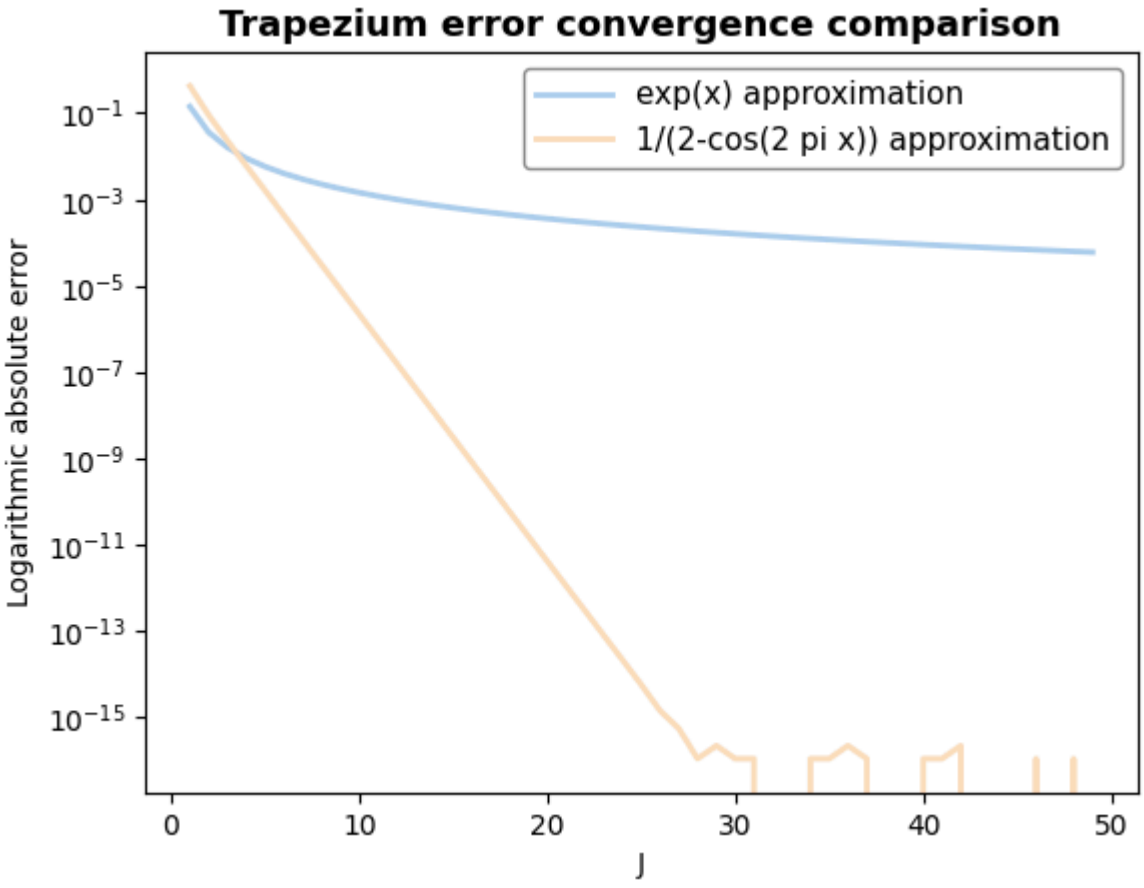
        I_exp = non_per_trap(lambda x: np.exp(x), j)
        I_cos = per_trap(lambda x: 1 / (2 - np.cos(2 * np.pi * x)), j)

        errs_exp.append(abs(I_exp - quad(np.exp, 0, 1)[0]))

        real_cos_I = quad(lambda x: 1 / (2 - np.cos(2 * np.pi * x)), 0, 1)
        errs_cos.append(abs(I_cos - real_cos_I)[0])

plt.semilogy(J, errs_exp, label="exp(x) approximation", \
        color="#abcdef", linewidth=2)
plt.semilogy(J, errs_cos, \
        label="1/(2-cos(2 pi x)) approximation", color="#fedcba", linewidth=2)
plt.xlabel("J"); plt.ylabel("Logarithmic absolute error");
plt.title(f"Trapezium error convergence comparison", fontweight="bold", \
        fontsize=13)
plt.legend(framealpha=0.9, \
        edgecolor='grey', fontsize=11,handlelength=2.5); plt.show()

print()
```




## Main features observed

- The log(absolute error) of $e^x$ converges to roughly $10^{-3}$ as J increases, with the shoulder of the curve occuring early on

- The log(absolute error) of the cos-related function decreases massively compared to our exponential function, with errors as low as $10^{-15}$ as J approaches 40.
- As the error reaches $10^{-16}$, strange oscillations occur on the graph, likely due to precision/rounding errors within Python. As I'm on a 64-bit machine, any number smaller than $10^{-16}$ cannot be accuractely represented on my machine. This will likely be the same across most devices.

In [5]:
```
import sys

print("Machine epsilon:", sys.float_info.epsilon)
```

```
Machine epsilon: 2.220446049250313e-16
```

# B2

$$T(e^{2k\pi ix}) = \frac{1}{J}\sum_{j=0}^{J-1} e^{2k\pi ij/J}$$

If $k \cong 0 \mod J$, then $k = mJ$ for some $m$, and thus it becomes:

$$\begin{aligned} T(e^{2k\pi ix}) &= \frac{1}{J}\sum_{j=0}^{J-1} e^{2mJ\pi ij/J} \\ &= \frac{1}{J}\sum_{j=0}^{J-1} e^{2m\pi ij} \\ &= \frac{1}{J}\sum_{j=0}^{J-1} 1 \\ &= 1 \end{aligned}$$

Otherwise, $k \ncong 0 \mod J$, and $T(u)$ becomes the sum of the J-th roots of unity, which we prove below to sum to 0.

Each J-th root of unity forms a geometric series $1 + \omega + \omega^2 + \ldots + \omega^{J-1}$ with common ratio $\omega$. Since $\omega^J = 1$,

$$\begin{aligned} \sum_{j=0}^{J-1} e^{2j\pi i/J} &= \frac{1-\omega^J}{1-\omega} \\ &= \frac{1-1}{1-\omega} \qquad = 0 \ \square \end{aligned}$$

From a proof in **A1**,

$$\int_0^1 e^{2\pi kix}\, dx = 0$$

Now, to provide an expression for $T(u) = \frac{1}{J}\sum_{j=0}^{J-1}\sum_{k=-\infty}^{\infty} u_k e^{2\pi kix}$,

The inner sum converges according to the preamble for Section A, with the outer sum clearly finite. Thus, we swap the sums as follows, breaking the sum into $k|J$ and $k \nmid J$:

$$
\begin{aligned}
T(u) &= \sum_{k=-\infty}^{\infty} u_k \frac{1}{J} \sum_{j=0}^{J-1} e^{2\pi kix} \\
&= \sum_{k=mJ} (u_k \frac{1}{J} \sum_{j=0}^{J-1} e^{2\pi kix}) + \sum_{k \neq mJ} (u_k \frac{1}{J} \sum_{j=0}^{J-1} e^{2\pi kix}) \\
&= \sum_{k=mJ} (u_k \times 1) + \sum_{k \neq mJ} (u_k \times 0) \\
&= \sum_{k=mJ} u_k
\end{aligned}
$$

It can also be easily shown that:

$$
\int_0^1 u(x)\, dx = \int_0^1 u(x) e^0\, dx = u_0
$$

:

The error can be finally simplified to:

$$
\begin{aligned}
T(u) - \int_0^1 u(x)\, dx &= \left( \sum_{k=mJ} u_k \right) - u_0 \\
&= \sum_{m \neq 0} u_{mJ} \quad \square
\end{aligned}
$$

# B3

From the lecture notes, we know the theoretical upper bound on composite trapezium rules is

$$
\begin{aligned}
\left| \int_a^b f(x)\, dx - Q_{1,J}^{[a,b]} \right| &= -\frac{1}{12} \sum_{j=1}^{J} h_j^3 f^{(2)} \zeta_j \\
&\leq \frac{b-a}{12} ||f^{(2)}||_{\infty,[a,b]} h^2
\end{aligned}
$$

In the case of $f(x) = exp(x)$,

$$
\begin{aligned}
\left| \int_0^1 e^x\, dx - Q_{1,J}^{[0,1]} \right| &\leq \frac{1}{12} e^1 h^2 \\
&\leq \frac{C}{J^2}
\end{aligned}
$$

This can explain the average convergence, with the error upper bound being $O(\frac{1}{J^2})$, however we've seen in lectures that much of the time the error ends up converging faster than its theoretical upper bound.

Now when looking at our function $u(x) = \frac{1}{2 - cos(2\pi x)}$, we can see this is periodic on [0,1]
With this, we can begin to apply an argument from *Proof of Theorem 3.1 by Fourier Series and aliasing* **[1]**, which uses similar ideas to those in **B2**. This text was found at the end of Lecture 12's Python PDF.

Beginning with the error formula proved in B2:

$$|T(u) - I| = \left| \sum_{m=\infty}^{\infty} u_{mJ} - I \right|$$
$$= \left| \sum_{m \neq 0} u_{mJ} \right|$$

Section 3 of **[1]** states that a periodic, analytic function will have exponentially decaying Fourier coefficients, with coefficients $u_j \leq Me^{-ja}$ for some constant $M, a$

To prove that $u(x)$ is analytic, I will use some closure properties ($p_1, p_2, p_3$) of analytic functions **[2]**. If $f(x)$ is analytic, then so is:

- $p_1 : f(x) + \alpha$
- $p_2 : \alpha f(x)$
- $p_3 : g(f(x))$

I start with $f(x) = x$, which is analytic from the section *Examples* in **[2]**, which states all polynomials are analytic.

Using $p_2$, $2\pi x$ is analytic too.

**[2]** States that all elementary trigonometric functions are analytic, so $cos(2\pi x)$ is too by $p_3$.

It is now reasonable to see that $u(x) = \frac{1}{2 - cos(2\pi x)}$ must then be analytic, with $p_1,\ (\alpha_1 = 2,\ \alpha_2 = -1)$ and $p_3,\ (g(x) = 1/x)$

Thus, we can bound the error by

$$\left| \sum_{m \neq 0} u_{mJ} \right| \leq \sum_{m \neq 0} |u_{mJ}|$$
$$\leq \sum_{m \neq 0} |Me^{-ma}|$$
$$|T(u) - I| \leq \frac{M}{e^{2\pi aJ} - 1}$$

This theoretical upper bound on the error decays exponentially in J, explaining the huge difference in magnitude of the errors between $1/(2 - cos(2\pi x))$ and $e^x$

References


- **[1]** https://epubs.siam.org/doi/pdf/10.1137/130932132
- **[2]** https://en.wikipedia.org/wiki/Analytic_function


# Section C

## C1

In [6]:
```
import matplotlib.image as mpimg
import matplotlib.pyplot as plt
```

```
import skimage as sk
import numpy as np

# Note -> I've had to truncate a lot of lines using \
# as the PDF doesn't wrap text within code blocks

# For the image gray-scaling:
# https://www.geeksforgeeks.org/python/python-grayscaling-of-images-using-opencv/

# For research on compressed image quality metrics:
# https://www.mathworks.com/help/images/image-quality-metrics.html
# https://sumn2u.medium.com/quality-metrics-in-image-compression-f6de68df1b8

def mse(A, C): return np.mean((A-C)**2)

def ssim(A, C):
        score, _ = sk.metrics.structural_similarity(A, C, full=True,
                data_range=(max(A.max(), C.max()) - min(A.min(), C.min())))
        return score

def compress(A, J):

        # Expect A to be a grayscale image i.e. [[int]]

        n, m = A.shape

        A_hat = np.fft.fft2(A)
        C_hat = np.zeros_like(A, dtype=complex)

        # Predicate to see if the Fourier coefficient should be kept or zeroed
        pred = lambda i, j: \
                (i < J and j < J) or \
                (i < J and j >= m - J) or \
                (i >= n - J and j < J) or \
                (i >= n - J and j >= m - J )

        for i in range(n):
                for j in range(m):
                        C_hat[i][j] = A_hat[i][j] if pred(i,j) else 0

        C = np.real(np.fft.ifft2(C_hat))
        mse_compressed, ssim_compressed = mse(A, C), ssim(A,C) # Calculate metrics
        return C, round(mse_compressed,1) , round(ssim_compressed,3)

mo_img = mpimg.imread("image_mo.jpg")

# Matrix multiply weights with the original m,n,3 matrix to get an m,n matrix,
# which can be displayed as grayscale with cmap="gray"

# Optimal weights taken from above reference
weights = np.array([0.2989, 0.5870, 0.1140])
mo_img = mo_img @ weights

print("Grey scaled shape:", mo_img.shape)

J = [20,50,100,250]

for j in J:
        compressed_img, err, qual = compress(mo_img, j)

        # Take a snapshot of Mo's head to highlight any artifacts more closely.
        # Would do a further zoom in, but don't want to take up too much space
```

```
        small_compressed_img = compressed_img[100:350, 450:650]

        # Display Mo reconstructed

        plt.imshow(compressed_img, cmap="gray") # Display grayscale
        ratio = round(4*j**2 / (mo_img.shape[0]*mo_img.shape[1]), 4)
        plt.title(f"J = {j}, Ratio: {ratio} , MSE = {err}, Quality = {qual}/1")
        plt.axis("off")
        plt.show()

        fig, axes = plt.subplots(1, 2, figsize=(5, 4))
        fig.suptitle("Zoomed in comparison", fontweight="bold",
                         fontsize=15)

        # Display Mo's face reconstructed, next to original

        # Reconstructed:
        axes[0].imshow(small_compressed_img, cmap="gray")
        axes[0].set_title(f"J = {j}, Ratio: {ratio},\n MSE = {err}, Quality = {qual}/1")
        axes[0].axis("off")

        # Original:
        axes[1].imshow(mo_img[100:350, 450:650], cmap="gray")
        axes[1].set_title(f"Original")
        axes[1].axis("off")
        plt.show()

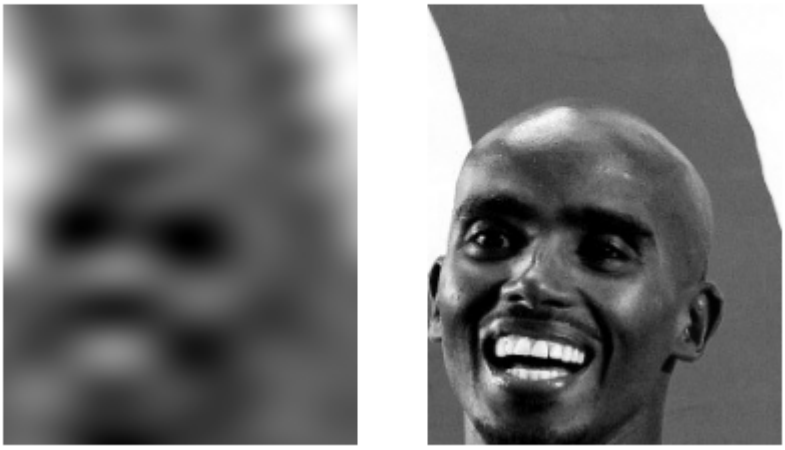
plt.imshow(mo_img, cmap="gray") # Display original photo for comparison
plt.title("Original photo"); plt.axis("off"); plt.show()
```

```
Grey scaled shape: (750, 1250)
```



## Zoomed in comparison

J = 20, Ratio: 0.0017,
MSE = 1369.1, Quality = 0.566/1 Original

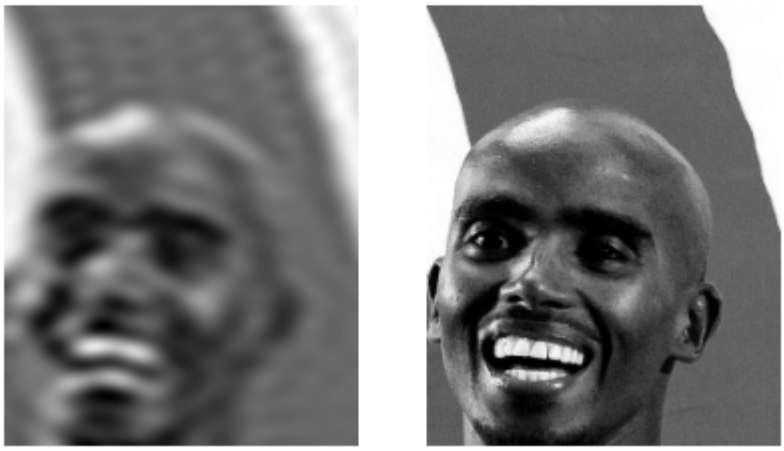


J = 50, Ratio: 0.0107 , MSE = 548.8, Quality = 0.674/1



## Zoomed in comparison

J = 50, Ratio: 0.0107,
MSE = 548.8, Quality = 0.674/1 Original



J = 100, Ratio: 0.0427 , MSE = 229.9, Quality = 0.792/1

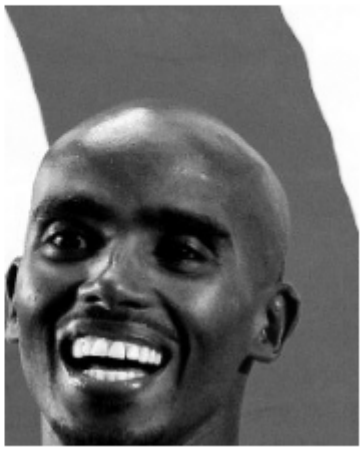



## Zoomed in comparison

J = 100, Ratio: 0.0427,
MSE = 229.9, Quality = 0.792/1



Original



J = 250, Ratio: 0.2667 , MSE = 63.6, Quality = 0.915/1






In the above plots:

- *J* is the side length of the square corner coefficients taken
- *Ratio* is the number of Fourier coefficients kept divided by original amount of coefficients = $\frac{4J^2}{n \times m}$
- *MSE* calculates the mean squared point-wise error between the original and compressed image
- SSIM (Structural Similarity) is used to obtain a score $\in [0, 1]$ detailing the quality of the compressed image. The closer to 1, the better matched the two images are

I found:

- As J increases, the compressed images become less blurry and reveal more edges, shadows, and fine detail. This is because the four $J \times J$ corner blocks of $\hat{A}$ contain the highest-frequency Fourier coefficients. When J is small, only a tiny fraction of these high-frequency components are kept, so the reconstruction shows only an outline of the image. As J grows, more high-frequency information is retained, improving sharpness and detail in the reconstructed image.
- The main difference between the $J = 250$ reconstruction and the original image is the lack of shadow/colour depth, not necessarily a blur effect like seen in smaller $J$

- One of many artifacts shown is the complete lack of detail of Mo's eye in the $J = 20$ example. In place of his eyes, there exists just a black hole. The embedding of the details of Mo's eyes must've not been within any of the $20 \times 20$ corner squares, as there is absolutely no information representing them shown. As J increases to 50, we can start to see his eye shape, eyebrows and eyelids start to develop
- The included metrics clearly improve as J increases and are given as a sanity check mostly. The more interesting metric is SSIM, which tell us that with $J = 250$, the compression recovers about 90% of the quality of the image, whilst only needing $0.267 \approx \frac{1}{4}$ of the Fourier coefficients i.e. information.

# C2

In [9]:
```
# Note -> I've had to truncate a lot of lines using \
# as the PDF doesn't wrap text within code blocks

def compress_v2(im, J, D):

        C = np.zeros_like(im)
        n, m = im.shape
        total_coefficients = 0

        for i in range(0, n, D):
                for j in range(0, m, D):

                        # Calculate next top left corner, accounting for edges
                        next_i = min(i+D, n)
                        next_j = min(j+D, m)

                        small_img = im[i:next_i, j:next_j]
                        C_small, _, _ = compress(small_img, J)
                        C[i:next_i, j:next_j] = C_small
                        max_coeffs = C_small.shape[0] * C_small.shape[1]
                        total_coefficients += min(max_coeffs, 4*(J**2))
        return C, total_coefficients

mo_img = mpimg.imread("image_mo.jpg")

weights = np.array([0.2989, 0.5870, 0.1140])
mo_img = mo_img @ weights

D, J = [20,75,150,300], [2,10,50,75]


for d,j in zip(D,J):

        compressed_img, total_coeff = compress_v2(mo_img, j, d)
        ratio = round(total_coeff/937500, 3)
        plt.imshow(compressed_img, cmap="gray")
        plt.title(f"D = {d}, J = {j}, Total coefficients: {total_coeff}, Ratio: {ratio}")
        plt.axis("off"); plt.show()

plt.imshow(mo_img, cmap="gray")
plt.title("Original image"); plt.axis("off"); plt.show()
```

D = 20, J = 2, Total coefficients: 38304, Ratio: 0.041




D = 75, J = 10, Total coefficients: 68000, Ratio: 0.073




D = 150, J = 50, Total coefficients: 437500, Ratio: 0.467









My choices ($C_j$) for D and J and their qualitative results:

- $C_1 : D = 20, J = 2$. This was chosen to show what it would look like with many small grids, with a small J relative to 20.
  As shown, many small grids are visible, with this reconstruction unlikely to be chosen over one of similar compression ratio from the previous algorithm. The artifacts made by the grid are quite visually unappealing, and are subjectively worse than just a blur.

- $C_2 : D = 75, J = 10$. I wanted to increase D while still maintaining J quite low respectively.
  The difference in quality between this plot and $C_1$ is huge, with there being much less blur whilst having a similarly low compression ratio of 0.073 versus 0.041. The grid problem still persists however, with it almost being more apparent than in $C_1$, due to the squares getting bigger and more detailed in their outline.

- $C_3 : D = 150, J = 50$ and $C_4 : D = 300, J = 75$. I'll keep these two combinations together, as they are very similar. In both, I wanted high-quality images, with $C_3$ having a higher proportion $J$ w.r.t $D$. In both plots, the square grids are almost unnoticeable, with very faint outlines due to the

higher quality of both images. Despite the lower compression ratio out of the two, I'd award $C_4$ the best quality, due to its slightly better colour depth and fainter grid lines.

Theoretically, the *improved* algorithm should keep more valuable information, as instead of completely ignoring the coefficients that aren't in the corners, it would create a more holistic reconstruction, taking coefficients from a more varied space of the input image.

With the data above, I would recommend to use the first algorithm in Exercise **C1**, as it is less complex, requires less customisation (1 parameter) and doesn't show grid outlines no matter the quality.